AnomalyLLM: Few-shot Anomaly Edge Detection for Dynamic Graphs using Large Language Models

Shuo Liu[1,2], Di Yao[1,*], Lanting Fang[5], Zhetao Li[4], Wenbin Li[1,2], Kaiyu Feng[3], Xiaowen Ji[5], Jingping Bi[1,*]
[1]*Institute of Computing Technology, Chinese Academy of Sciences, China*
[2]*University of Chinese Academy of Sciences, China*
[3]*Beijing Institute of Technology, China*
[4]*Xiangtan University, China*
[5]*Southeast University, China*
{liushuo22s,yaodi,liwenbin20z,bjp}@ict.ac.cn, lantingf@outlook.com, liztchina@hotmail.com,
fengky@bit.edu.cn, jxiaowen@126.com

2024 IEEE International Conference on Data Mining (ICDM) | 979-8-3315-0668-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICDM59182.2024.00093

***Abstract*—Detecting anomaly edges for dynamic graphs aims to identify edges significantly deviating from the normal pattern and can be applied in various domains, such as cybersecurity, financial transactions and AIOps. With the evolving of time, the types of anomaly edges are emerging and the labeled anomaly samples are few for each type. Current methods are either designed to detect randomly inserted edges or require sufficient labeled data for model training, which harms their applicability for real-world applications. In this paper, we study this problem by cooperating with the rich knowledge encoded in large language models(LLMs) and propose a method, namely AnomalyLLM. To align the dynamic graph with LLMs, AnomalyLLM pre-trains a dynamic-aware encoder to generate the representations of edges and reprograms the edges using the prototypes of word embeddings. Along with the encoder, we design an in-context learning framework that integrates the information of a few labeled samples to achieve few-shot anomaly detection. Experiments on four datasets reveal that AnomalyLLM can not only significantly improve the performance of few-shot anomaly detection, but also achieve superior results on new anomalies without any update of model parameters.**

***Index Terms*—Dynamic Graphs, Anomaly Detection, Few-Shot Learning, Large Language Models**

I. Introduction

The dynamic graph is a powerful data structure for modeling the evolving relationships among entities over time. Anomaly edges in dynamic graphs, which refer to the unexpected or unusual relationships between entities, are valuable traces of almost all web applications. Due to the temporary nature of dynamics, the types of anomaly edges vary greatly, leading to the difficulty of acquiring sufficient labeled samples of new types. Therefore, detecting anomaly edges with few labeled samples plays a vital role in dynamic graph analysis and is of great importance for various applications, including network intrusions, financial fraud detection [1], and *etc.*

Recently, various techniques have been proposed to detect anomalies in dynamic graphs. Supervised methods [2] require obtaining a sufficient number of labeled anomalous edges for model training, which is a challenge for dynamic graphs, limiting their scalability. Unsupervised methods [3]–[7] are mainly designed to detect randomly inserted edges as anomalies and are difficult to extend to other anomaly types. Only SAD [8] tries to address the problem using semi-supervised learning. However, the training data used in SAD contains hundreds of labeled samples, which is also impractical in most cases. As shown in Figure 1, with the evolution of time, the anomaly edges may change and new types of anomaly edges would emerge. Thus, the problem we aim to solve is to identify various types of anomaly edges in the dynamic graph with few labeled samples (less than 10) for each type.

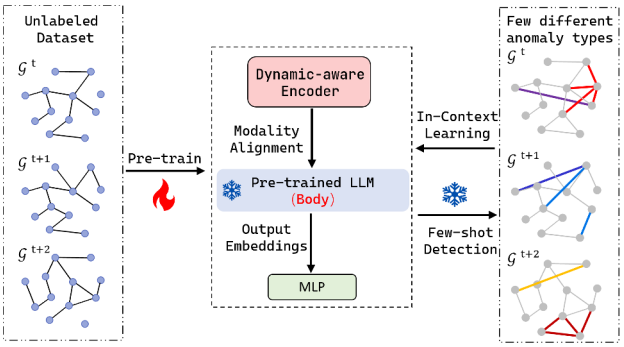


Fig. 1. The motivation of AnomalyLLM. In the real world, edge anomaly types are diverse, evolving, and typically associated with limited labeled data.

With the rapid progress of foundation models, large language models (LLMs) show a remarkable capability of understanding graph data and generalizability on new tasks, which offers a promising path to achieve few shot anomaly edges detection for dynamic graphs. However, this task is also challenging in three aspects: (1) Representation of dynamic graph. Anomaly edges in dynamic graph are related to the changing of the graph topology. The edge representations should not only encode the information of adjacent topology but also be aware of the temporal dynamics. (2) Alignment between graph and neural language. LLMs operate on discrete tokens, whereas dynamic graphs change in continuous time. It remains an open challenge to align the semantics between dynamic graphs and LLMs. (3) Adaptation with few anomaly samples. To achieve few-shot detection, both LLMs and the

*Corresponding authors.

2374-8486/24/$31.00 ©2024 IEEE
DOI 10.1109/ICDM59182.2024.00093

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.

anomaly detector should make full use of the label information of limited anomaly samples to identify different anomalies.

To solve the challenges, we proposed AnomalyLLM to integrate the power of LLMs and detect anomaly edges with few labeled samples. Compared to existing solutions, AnomalyLLM has the following attractive advantages: (1) **Anomaly type-agnostic.** AnomalyLLM conducts the dynamic graph encoding and the modality alignment in an unsupervised manner. The information of anomaly type is only used to construct the prompt of in-context learning. For detecting different anomaly types, all we need is a new prompt, *i.e.* the model parameters are anomaly type-agnostic. (2) **Fine-tuning free.** The parameters in LLMs do not require expensive fine-tuning computations. (3) **Simple to upgrade.** LLMs are only related to modality alignment parameters, and the training time for these parameters is not lengthy. If there is an alternative more powerful LLM, AnomalyLLM is simple to be upgraded by retraining the related parameters. The main contributions can be summarized as follows:

- We propose a novel method leveraging the advanced capabilities of LLMs for few-shot anomaly edge detection.
- We introduce a reprogramming-based modality alignment technique, which represents the graph edge embeddings with some text prototypes, to bridge the gap between the dynamic-aware encoder and the LLMs.
- An in-context learning strategy is designed to integrate the information of a few labeled samples, making AnomalyLLM adaptable to various anomaly types with minimal computational overhead.

II. Related Work

A. Graph Anomaly Detection.

Most supervised methods [2] rely on labeled data to train anomaly detectors, which may result in poor performance due to the limited number of samples in real-world scenarios. Unsupervised methods [3]–[7] primarily identify anomalies based on statistical measures or graph topology, mainly rely on randomly-inserted edges [5] during training, which differs from actual anomalies. Recently, SAD [8] have been proposed to incorporate both labeled and unlabeled data. However, these methods rely on a considerable amount of labeled samples.

B. Large Language Models.

The emergence of large language models [9] has ushered in a new era of few-shot learning capabilities, exemplified by their application in In-Context reasoning with minimal examples. Many LLM-based methods [10] are proposed to graph analysis, primarily focusing on leveraging the rich textual attributes inherent in graphs. These techniques mainly rely on modality alignment between graph representations and textual properties. However, this reliance significantly limits their applicability in scenarios where textual attributes are absent. While some efforts [11], [12] have been made to enhance LLMs' understanding of non-textual data like time-series, through reprogramming techniques, the application of these methodologies to dynamic graphs data remains unexplored.

C. Few-shot Learning in Dynamic Graphs.

The challenge of limited labeled data is pervasive in real-world applications. Many studies have explored for few-shot learning, using techniques like meta-learning or contrastive learning [13]. However, these methods are generally tailored to static graphs or specific tasks [14], leaving a gap in anomaly edge detection for dynamic graphs.

III. Preliminary

A. Problem Definition

Let $\mathcal{G} = [\mathcal{G}^1, ..., \mathcal{G}^t, ..., \mathcal{G}^T]$ denote a sequence of graph snapshots spanning timestamps 1 to $\mathcal{T}$, where each snapshot $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t)$ represents the state of the graph at time t with $\mathcal{V}^t$ being the set of nodes and $\mathcal{E}^t$ the set of edges. An edge $e^t_{i,j} = (v^t_i, v^t_j) \in \mathcal{E}^t$ signifies an interaction between nodes v^t_i and v^t_j at time t. Given a specified anomaly type $\mathcal{T}$ and related set of few anomaly edges $\mathcal{E}_{\mathcal{T}} = \{\mathcal{T}_1, \cdots, \mathcal{T}_a\}$, where a is the number of anomaly edges, our objective is to detect whether edge $e^t_{i,j}$ in $\mathcal{G}^t$ is an anomaly edge of type $\mathcal{T}$ or not.

IV. Methodology

As shown in Figure 2, AnomalyLLM consists of three key modules. We specify the details of each module respectively.

A. Dynamic-aware Contrastive Pretraining

1) Dynamic-aware Encoder: Given an edge $e^t_{i,j}$, we first construct subgraphs $\mathcal{S}^t_{i,j}$, then fed it into the subgarph-based edge encoder to obtain the edge representation $\mathbf{r}^t_{i,j}$.

Structural-Temporal Subgraph Construction. For an edge $e^t_{i,j}$, we design to construct structural-temporal subgraphs for both source and target nodes. Given an edge $e^t_{i,j} = (v^t_i, v^t_j) \in \mathcal{E}^t$, we first construct a diffusion matrix [5] $\mathbf{D}^t \in \mathbb{R}^{N\times N}$ of $\mathcal{E}^t$ to select the structure context, where N represents the number of nodes in $\mathcal{E}^t$.

Each row d^t_i of $\mathbf{D}^t$ indicates the connectivity strength of the $i-th$ node with all other nodes in the graph $\mathcal{G}^t$. For $e^t_{i,j} = (v^t_i, v^t_j)$, we utilize d^t_i and d^t_j to select the most significant top-K adjacent nodes of $\mathcal{V}^t$ to form $\mathcal{V}^t_i$ and $\mathcal{V}^t_j$ as the subgraph nodes of the source node v^t_i and target node v^t_j respectively. Then, we link the nodes in $\mathcal{V}^t_i$ to its related node v^t_i to generate $\mathcal{E}^t_i$ and obtain the subgraphs $g^t_i = \{\mathcal{V}^t_i, \mathcal{E}^t_i\}$. Similar operations are conducted for the target node v^t_j to obtain $g^t_j = \{\mathcal{V}^t_j, \mathcal{E}^t_j\}$. In this way, both the source and the target in $e^t_{i,j}$ can be represented by the relevant surrounding subgraphs $g^t_{i,j} = [g^t_i, g^t_j]$.

To obtain the temporal context of $e^t_{i,j}$, AnomalyLLM utilizes a sliding window Γ to filter a sequence of graph slices $\mathcal{G}^\Gamma_t = \{\mathcal{G}_{t-\Gamma+1}, \ldots, \mathcal{G}_t\}$. For each graph slice, we use the described method to construct subgraphs. Therefore, we can construct a sequence of subgraph $\mathcal{S}^t_{i,j} = \{g^\tau_{i,j}\}$ for $\tau = t-\Gamma+1, \ldots, t$. $\mathcal{S}^t_{i,j}$ contains not only the structure but also the temporal context of $e^t_{i,j}$. The representation of $\mathcal{S}^t_{i,j}$ can be used to detect the anomaly in $\mathcal{G}$.

Subgraph-based Edge Encoder. Given the subgraph sequence $\mathcal{S}^t_{i,j}$ of edge $e^t_{i,j}$, we feed them into the subgraph-based edge encoder which synergizes the Transformer and Graph

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.

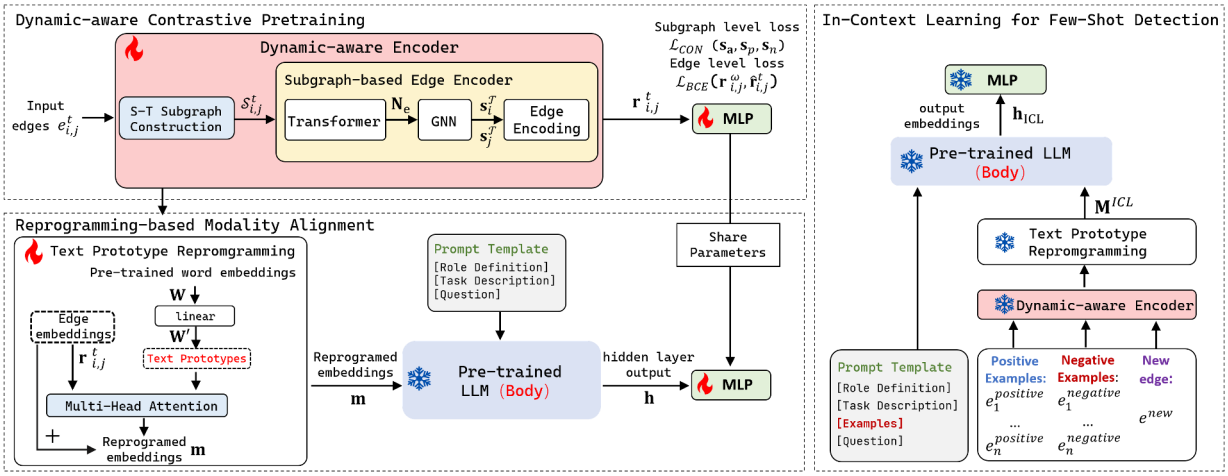


Fig. 2. Overview of AnomalyLLM. AnomalyLLM comprises three modules: Dynamic-aware Contrastive Pretraining, Reprogramming-based Modality Alignment, and In-Context Learning for Few-Shot Detection.

Neural Network (GNN) models to obtain edge representation $\mathbf{r}^t_{i,j} \in \mathbb{R}^{d_m}$, where d_m represents the embedding dimension. Following the same setting as TADDY [5], we assume the nodes in $\mathcal{G}^\circ$ are stable and conduct the following four steps on the input $\mathcal{S}^t_{i,j}$:

- **Node Encoding.** For each node v^τ_l in every g^τ_i within $\mathcal{S}^t_{i,j}$, we construct the node encoding using three aspects, *i.e.*, $\mathbf{z}_l = \mathbf{z}_{\text{diff}}(v^\tau_l) + \mathbf{z}_{\text{dist}}(v^\tau_l) + \mathbf{z}_{\text{temp}}(v^\tau_l) \in \mathbb{R}^{d_{enc}}$. Here, $\mathbf{z}_{\text{diff}}(v^\tau_l)$ represents the diffusion-based spatial encoding capturing the global structural role of node v^τ_l, $\mathbf{z}_{\text{dist}}(v^\tau_l)$ denotes the distance-based spatial encoding, reflecting the local structural context; and $\mathbf{z}_{\text{temp}}(v^\tau_l)$ provides the relative temporal information of node v^τ_l which is the same for all nodes at the time slice τ.
- **Temporal Encoding.** We model the temporal information of nodes in $\mathcal{S}^t_{i,j}$ by reorganizing the node encoding into an encoding sequence $\mathbf{Z}_e = [[\mathbf{z}_l]_{v_l \in g^\tau_{i,j}}]_{g^\tau_{i,j} \in \mathcal{S}^t_{i,j}}$, with the dimension of $\mathbf{Z}_e$ being $\mathbb{R}^{(2(K+1)\cdot\Gamma)\times d_{enc}}$. We feed $\mathbf{Z}_e$ into a vanilla Transformer block to obtain the node embeddings $\mathbf{N}_e = \text{Transformer}(\mathbf{Z}_e)$. The dimension of node embedding, $denc$, is specified here.
- **Subgraph Encoding.** We employ GNN to generate the graph representations of all related subgraphs in $\mathcal{S}^t_{i,j}$. For each subgraph g^τ_i, we extract the related node embeddings $\mathbf{N}^t_i \in \mathbb{R}^{(K+1)\times d_{enc}}$ from $\mathbf{N}_e$ and utilize GNN to obtain the embedding of node v^τ_l as the subgraph embedding $\mathbf{s}^\tau_i$. To fuse the information on different timesteps, we stack the Γ embeddings of v^τ_l to generate $\mathbf{s}^\tau_i \in \mathbb{R}^{(K+1)\times d_{enc}}$
- **Edge Encoding.** To obtain the representation of $e^t_{i,j}$, we first conduct average pooling on the related subgraph embeddings $\mathbf{s}^\tau_i$ and $\mathbf{s}^\tau_j$. Then we concatenate the resulting vectors and project the concatenated vector into the LLM's hidden dimension d using a fully connected layer.

2) Contrastive Learning-based Optimization: AnomalyLLM employs contrastive learning to optimize the parameters in the dynamic-aware encoder. To obtain negative samples and achieve anomaly detection, we follow two principles in sampling: (1) edges with different subgraphs of related nodes should not have similar embeddings; (2) the embeddings between existing edges and randomly sampled edges should be distinguishable.

For edge $e^t_{i,j}$, we check its adjacent graphs, $\mathcal{G}^{t-1}$ and $\mathcal{G}^{t+1}$. The sampling should include two levels, *i.e.* edge level and subgraph level. As shown in Figure 3, we randomly sample a node $\hat{v}^\omega_{neg}$, where $\omega = t \pm 1$ not directly connected to v^ω_i and generate the edge embedding $\hat{\mathbf{r}}^\omega_{i,neg}$ for the edge $< v^\omega_i, \hat{v}^\omega_{neg} >$. At the edge level, we employ a Multilayer Perceptron (MLP) layer as the anomaly detector to identify whether the input edge is randomly sampled. Here, we feed the embeddings of $\mathbf{r}^\omega_{i,j}$ and $\hat{\mathbf{r}}^\omega_{i,neg}$ into the detector and employ binary cross-entropy loss to make them distinguishable:

$$\mathcal{L}_{BCE} = -\log(1 - \text{MLP}(\mathbf{r}^t_{i,j})) + \log(\text{MLP}(\hat{\mathbf{r}}^\omega_{i,neg}))$$

At the subgraph level, we consider the subgraph of $\hat{v}^\omega$ as the negative sample of subgraph of v^t_i and utilize the subgraph of v^ω_i in different timestamps as the positive sample to construct a triplet. As shown in Figure 3, we sample negative subgraph and contrastive training triplet for node v^ω_i. Since the edge embeddings are concatenations of subgraph embeddings, AnomalyLLM employ contrastive loss to enlarge the dissimilarity between subgraph embeddings, and the pre-training loss is the combination of both edge level loss and subgraph level loss.

$$\mathcal{L}_{con} = -\log \frac{\exp(\cos(\mathbf{s}_a, \mathbf{s}_p)/\delta)}{\exp(\cos(\mathbf{s}_a, \mathbf{s}_p)/\delta) + \exp(\cos(\mathbf{s}_a, \mathbf{s}_n)/\delta)} \quad (1)$$

$$\mathcal{L} = \mathcal{L}_{BCE} + \mathcal{L}_{con} \quad (2)$$

where $\mathbf{s}_a, \mathbf{s}_p, \mathbf{s}_n \in \mathbb{R}^{d_{emb}}$ represent the subgraph embeddings for the anchor, positive, and negative samples in the triplet.

B. Reprogramming-based Modality Alignment

AnomalyLLM employs LLMs as the backbone to enhance the generalization ability of edge embeddings output by the dynamic-aware encoder. We propose reprogramming-based modality alignment techniques to bridge the gap. For simplicity, we omit the subscript and note the edge embedding

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.

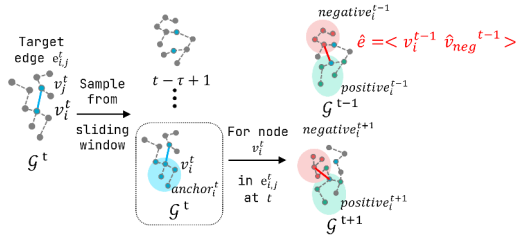


Fig. 3. Sample process of contrastive training triplet

with **r**. Taking the **r** as input, AnomalyLLM first reprograms it with the prototype of the word embeddings and feeds the reprogramed vector into LLMs to generate $\mathbf{h} \in \mathbb{R}^d$. Both **r** and **h** are fused as the final edge embedding to input to the LLM for anomaly detection.

1) Text Prototype Reprogramming: To leverage the capability of LLM for dynamic graph analysis, we first select a subset of word embeddings and cluster them as text prototypes for reprogramming edge embeddings. Given the pre-trained word embeddings of LLMs, we refine a subset of words $\mathbf{W} \in \mathbb{R}^{V \times d}$ related to dynamic graphs to generate text prototypes. The output words in different rounds are combined to obtain V related words. Based on these words, we construct the text prototype with liner transformation: $\mathbf{W}' = \mathbf{M} \cdot \mathbf{W}$, where $\mathbf{M} \in \mathbb{R}^{V' \times V}$ and V' is the number of prototypes. Given an edge embedding **r**, AnomalyLLM utilize multi-head cross-attention to conduct reprogramming. We use **r** as the query vector and employ $\mathbf{W}'$ as the key and value matrices. For each attention head c in $\{1, \ldots, C\}$, we compute the related query, key and value matrices, *i.e.*, $\mathbf{Q}_c$, $\mathbf{K}_c$ $\mathbf{V}_c$. The attention operation for each head is formalized as: $\mathbf{z}_c = \text{ATTENTION}(\mathbf{Q}_c, \mathbf{K}_c, \mathbf{V}_c)$.

The outputs from all heads are aggregated to obtain $\mathbf{z} \in \mathbb{R}^d$. We then add **z** to the edge embedding **r** to obtain the reprogramed representation $\mathbf{m} \in \mathbb{R}^d$ of the given edge $e_{i,j}^t$.

2) Pseudo Label for Anomaly Fine-tuning: The backbone LLM takes the reprogrammed representation **m** as input to generate the final representation for anomaly detection. We utilize the randomly sampled edges (detailed in Section IV-A2) as pseudo anomaly labels to fine-tune the parameters of the dynamic-aware encoder and anomaly detector.

As shown in Figure 4, we design a template of prompt for both alignment fine-tuning and in-context learning detection. The template consists of four aspects and the $<$ Edge $>$ is a mask token for the input edge embedding. The instruction is fed into the LLM and the hidden state of the $<$ Edge $>$ token is selected as the final representation vector of edge e. For conciseness, we use v to represent $v_{i,j}^t$. This procedure can be formalized as $\mathbf{H} = \mathbf{LLM}([\mathbf{u}, \mathbf{m}])$, where $\mathbf{u} \in \mathbb{R}^{L \times d}$ is the related embeddings of instruction templates and $\mathbf{H} \in \mathbb{R}^{(L+1) \times d}$ is the last hidden layer output of the LLM. We utilize the last position of **H**, *i.e.* **h** for detection. Note that our backbone LLM employs causal attention to compute **h**. Thus, for different edges, the front parts of **h** are the same. We can use this character to further reduce the computation workload in the pre-training procedure.

In this module, we reuse the MLP detector described in Section IV-A2 and replace the input edge embedding **r** with

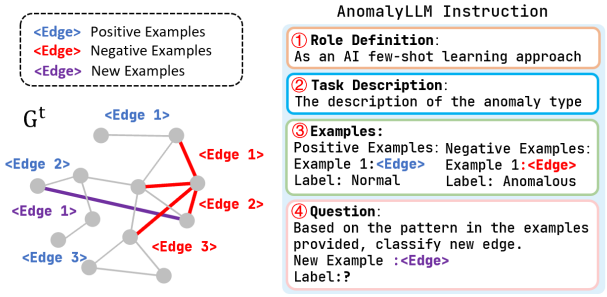


Fig. 4. The prompt of In-Context Learning

the reprogramed edge embedding **r**. The anomaly score for an edge e is computed with $f(e) = \text{MLP}(\mathbf{h})$. We also used the randomly selected edges $\hat{e}$ as negative samples and the existing edges as positive samples to construct pseudo labels. A binary cross-entropy (BCE) loss of pseudo labels is employed to optimize the parameters of the dynamic-aware encoder and the detector. Note that the MLP detector is optimized in both pre-training and alignment fine-tuning. In few-shot anomaly detection, the MLP detector cooperates with the in-context learning strategy to detect various types of anomalies. During the whole procedure, the parameters of LLM are intact.

C. In-Context Learning for Few-Shot Detection

Given a set of anomaly edges $\mathcal{E}_{\mathcal{T}} = \{\mathcal{T}_1, \cdots, \mathcal{T}_a\}$ of anomaly type $\mathcal{T}$, AnomalyLLM aim to detect whether the new edge e is an anomaly edge of $\mathcal{T}$ or not. Considering that the pretraining procedure of AnomalyLLM has no information about the anomaly type, we need to make full use of the labeled information of $\mathcal{E}_{\mathcal{T}}$. In this paper, we proposed to use in-context learning that encodes edges in $\mathcal{E}_{\mathcal{T}}$. Next, we introduce the construction of the prompt template and few-shot anomaly detection respectively.

1) Prompt Template Construction: As shown in Figure 4, the prompt first defines the role of LLM as a few-shot anomaly detector followed by the description of anomaly type $\mathcal{T}$. For the example part, we select the same number of edges $\mathcal{E}'/\mathcal{E}_{\mathcal{T}}^{'}$ from $\mathcal{E}/\mathcal{E}_{\mathcal{T}}$ as the normal and anomaly samples and generate the embedding of edges in $\mathcal{E}' \cup \mathcal{E}_{\mathcal{T}}^{'}$ with dynamic-aware encoder denoted by $\mathbf{M}^{ICL}$. These edges are then processed through the reprogramming module for modality alignment and to build the prompt examples.

$$\mathbf{M}^{ICL} = \{\mathbf{m}_1^{pos}, \ldots, \mathbf{m}_n^{pos}, \mathbf{m}_1^{neg}, \ldots, \mathbf{m}_P^{neg}, \mathbf{m}^{new}\}$$
$$\mathbf{h}_{ICL} = \mathbf{mean}(\mathbf{LLM}([\mathbf{u}_{ICL}, \mathbf{M}^{ICL}][:-1]))$$
$$f(e^{new}) = \text{MLP}(\mathbf{h}_{ICL})$$

where $\mathbf{m}_u^{pos}, \mathbf{m}_u^{neg} \in \mathbb{R}^{d_m}$ are the reprogrammed embeddings of the u-th positive and negative edge examples, respectively, and $\mathbf{m}^{new} \in \mathbb{R}^{d_m}$ is the reprogrammed embedding of the edge under investigation. In the prompt template, we employ mask token $<$ Edge $>$ to represent edge embeddings and each example has a related label tag. Given a new edge e^{new} needed to be detected, we conduct the same operations of examples to obtain the edge embedding.

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE COMPARISON RESULTS OF FEW-SHOT ANOMALY DETECTION ON MULTIPLE ANOMALY TYPES.

Dataset	Model	1-shot			5-shot			10-shot		
		CDA	LPL	HHL	CDA	LPL	HHL	CDA	LPL	HHL
BlogCataLog	StrGNN	0.5891	0.5756	0.5974	0.6018	0.6041	0.6122	0.6222	0.6329	0.6402
	AddGraph	0.5994	0.6023	0.5988	0.6097	0.6033	0.6104	0.6216	0.6238	0.6172
	Deep Walk	0.6102	0.6073	0.6202	0.6113	0.6122	0.6196	0.6155	0.6176	0.6154
	TGN	0.6732	0.6699	0.6919	0.7112	0.7023	0.7118	0.7263	0.7387	0.7311
	GDN	0.6733	0.6795	0.6609	0.6997	0.7051	0.7121	0.7321	0.7311	0.7319
	SAD	0.6841	0.6792	0.6411	0.7002	0.7018	0.6988	0.7342	0.7216	0.7265
	TADDY	0.6892	0.6983	0.6891	0.7148	0.7186	0.7177	0.7258	0.7326	0.7334
	AnomalyLLM	**0.8288**	**0.8334**	**0.8255**	**0.8331**	**0.8319**	**0.8407**	**0.8402**	**0.8456**	**0.8447**
UCI Message	StrGNN	0.6143	0.5956	0.5722	0.6113	0.7132	0.6512	0.6442	0.6724	0.6249
	AddGraph	0.5842	0.5466	0.5647	0.6018	0.6667	0.6321	0.4642	0.5728	0.7001
	Deep Walk	0.6198	0.6187	0.6142	0.6256	0.6263	0.6176	0.6255	0.6209	0.6197
	TGN	0.6521	0.6535	0.6643	0.7098	0.7193	0.7155	0.7335	0.7365	0.7324
	GDN	0.6577	0.6818	0.6611	0.7201	0.7289	0.7255	0.7493	0.7511	0.7546
	SAD	0.6703	0.6587	0.6693	0.7102	0.7146	0.7194	0.7416	0.7453	0.7406
	TADDY	0.6992	0.7078	0.6132	0.7204	0.7237	0.7218	0.7255	0.7278	0.7243
	AnomalyLLM	**0.8414**	**0.8358**	**0.8368**	**0.8446**	**0.8459**	**0.8424**	**0.8488**	**0.8546**	**0.8442**

2) Few-shot anomaly detection: For a specific anomaly type $\mathcal{T}$, the ICL template can be constructed in advance. Assuming e^{new} is a new edge to be detected, AnomalyLLM utilize the dynamic-aware encoder to obtain an intermediate vector and reprogram it with text prototypes. By embedding the reprogrammed vector into the ICL template, we obtain the input of LLM to generate the edge embedding $\mathbf{h}_{ICL}$. Then, the edge embedding is fed into the pre-trained anomaly decoder, *i.e.* the MLP layer, to calculate the probability of e^{new} to be an anomaly of $\mathcal{T}$. For different anomaly types, we can build multiple ICL templates by using a few labeled samples.

V. EXPERIMENTS

A. Experimental Settings

1) Data Descriptions: We use four datasets to evaluate the performance of AnomalyLLM. The main experiments are conducted on two widely-used benchmark datasets, *i.e.*, UCI Messages [16] and Blogcatalog [15]. To evaluate the performance of AnomalyLLM on real-world anomaly detection task and test the capability of AnomalyLLM, we also employ two datasets with real anomaly, *i.e.* T-Finance and T-Social [17].

2) Experimental Protocol: We follow the experiments of [18] and generate three kinds of systematic anomaly types, *i.e.*, Contextual Dissimilarity Anomaly(CDA), Long-Path Links (LPL) and Hub-Hub Links(HHL) for UCI Messages and Blogcatalog datasets. We build 1-shot, 5-shot and 10-shot labeled edges for each anomaly type to obtain the AUC results.

3) Baselines: We compare AnomalyLLM with seven baselines. For the graph representation method, we select **DeepWalk** [19] to generate the representations of edges. For unsupervised method, we employ the recent three works, *i.e.* **StrGNN** [4], **AddGraph** [6], and **TADDY** [5], as our baselines. For semi-supervised methods, we use **GDN** [2], **TGN** [20]and **SAD** [8] for performance comparison.

4) Hyperparameters setting: For subgraph construction, we set the number k to be 14 and Γ is 4. For edge encoder, the embedding dimension d is 512. We train all the datasets with 150 epochs. During the modality alignment, we fine-tune the encoder and anomaly detector for 20 epoch.

B. Performance Comparison

We compare AnomalyLLM against seven baselines and summarize the results in Table I. Overall, AnomalyLLM outperforms all baselines on all datasets. For different anomaly types, AnomalyLLM achieves stable improvements on CDA, LPL and HHL. As shown in Table I, the AUC of different anomaly types are over 80% indicating that AnomalyLLM is anomaly type-agnostic. Moreover, with the increase of labeled samples, the performances of both AnomalyLLM and baseline methods are improved steadily. Compared to the 10-shot setting, SAD's performance dropped significantly in the 1-shot setting, with a roughly 14% decrease in AUC, as it requires a larger number of labeled data for anomaly detection. In contrast, AnomalyLLM maintains an AUC over 0.82 on both datasets. We attribute this to the effectiveness of ICL module which excites the advanced capabilities of LLMs.

C. Efficiency Experiments

As shown in the left part of Figure 5, the fine-tuning time increases linearly according to the number of edge sizes. The inference time of ICL detection is shown in the right part of Figure 5. We can observe that the inference time of AnomalyLLM is comparable with other baselines under different batch sizes.

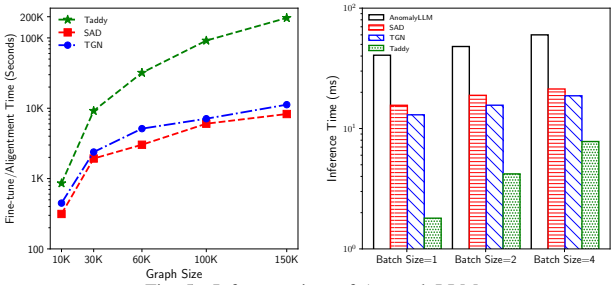


Fig. 5. Inference time of AnomalyLLM

D. Ablation Results:

We remove the proposed dynamic-aware encoder, the alignment training module and the ICL detection respectively, and obtain w/o encoder, w/o ICL and w/o ICL. As shown in Table

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ABLATION RESULTS

Dataset	Method	Anomaly Types		
		CDA	LPL	HHL
UCI Message	w/o ICL	0.7406	0.7465	0.7328
	w/o alignment	0.7849	0.7892	0.7994
	w/o encoder	0.7727	0.7883	0.7822
	AnomalyLLM	0.8402	0.8456	0.8447

TABLE III
PERFORMANCE ON REAL-WORLD LABELED DATASET

Dataset	Method	1-shot	5-shot	10-shot
T-Finance	AddGraph	0.6126	0.6149	0.6277
	GDN	0.6672	0.6689	0.6898
	SAD	0.6724	0.6754	0.6876
	AnomalyLLM	**0.8018**	**0.8056**	**0.8087**
T-Social	AddGraph	0.6116	0.6245	0.6221
	GDN	0.6694	0.6782	0.6908
	SAD	0.6779	0.6746	0.6805
	AnomalyLLM	**0.8101**	**0.8187**	**0.8206**

TABLE IV
SENSITIVITY ANALYSIS OF ANOMALYLLM W.R.T. DIFFERENT NUMBERS OF NODES IN EACH SUBGRAPH G_i^t ON THE BLOGCATALOG.

Number	2	6	10	14	18	20
AUC	0.7624	0.7896	0.8389	0.8456	0.8412	0.8442

II. Comparing the results of AnomalyLLM with w/o encoder, we observe the edge construction by focusing on subgraph embeddings from both sides can extract useful information and capture the evolving properties of edges in dynamic graphs. From the results of w/o ICL and AnomalyLLM, we can conclude that the ICL's capacity to efficiently utilize minimal labeled data is more effective than fine-tuning.

E. Performance on Real-World Labeled Dataset

We verify the performance on two real-world datasets, *i.e.*, T-Finance and T-Social, which have over 100 million edges. The results are summarized in Table III. Overall, AnomalyLLM outperforms all baselines on all datasets.

F. Sensitivity Analysis

To analyze the impact of different numbers of nodes in the Structural-Temporal Subgraph Sampler, we varied the node count from 2 to 20 and presented the average performance in Table IV. Notably, there was a significant performance boost when the node count reached 10, but performance exhibited a slight decline after reaching 14 nodes.

VI. CONCLUSION

In this paper, we are the first to integrate LLMs with dynamic graph anomaly detection, addressing the challenge of few-shot anomaly edge detection. AnomalyLLM leverages LLMs to effectively understand and represent the evolving relationships in dynamic graphs. AnomalyLLM reprograms the edge embedding to align the semantics between dynamic graph and LLMs. Moreover, an ICL strategy is designed to enable efficient and accurate detection of various anomaly types with a few labeled samples. Extensive experiments across multiple datasets demonstrate that AnomalyLLM not only significantly outperforms existing methods in few-shot settings but also sets a new benchmark in the field.

ACKNOWLEDGMENT

This work has been supported by the National Natural Science Foundation of China under Grant No. 62472405. This work is also sponsored by CCF-DiDi GAIA Collaborative Research Funds for Young Scholars.

REFERENCES

[1] X. Huang, Y. Yang, Y. Wang, C. Wang, Z. Zhang, J. Xu, L. Chen, and M. Vazirgiannis, "Dgraph: A large-scale financial dataset for graph anomaly detection," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 765–22 777, 2022.

[2] K. Ding, Q. Zhou, H. Tong, and H. Liu, "Few-shot network anomaly detection via cross-network meta-learning," in *Proceedings of the Web Conference*, 2021, pp. 2448–2456.

[3] J. Liu, F. Xia, X. Feng, J. Ren, and H. Liu, "Deep graph learning for anomalous citation detection," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 6, pp. 2543–2557, 2022.

[4] L. Cai, Z. Chen, C. Luo, J. Gui, J. Ni, D. Li, and H. Chen, "Structural temporal graph neural networks for anomaly detection in dynamic graphs," in *Proceedings of the 30th ACM international conference on Information & Knowledge Management*, 2021, pp. 3747–3756.

[5] Y. Liu, S. Pan, Y. G. Wang, F. Xiong, L. Wang, Q. Chen, and V. C. Lee, "Anomaly detection in dynamic graphs via transformer," *IEEE Transactions on Knowledge and Data Engineering*, 2021.

[6] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "Addgraph: Anomaly detection in dynamic graph using attention-based temporal gcn." *IJCAI*, vol. 3, 2019, p. 7.

[7] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proceedings of the 2019 SIAM International Conference on Data Mining*. SIAM, 2019, pp. 594–602.

[8] S. Tian, J. Dong, J. Li, W. Zhao, X. Xu, B. Song, C. Meng, T. Zhang, L. Chen *et al.*, "Sad: Semi-supervised anomaly detection on dynamic graphs," *arXiv preprint arXiv:2305.13573*, 2023.

[9] M. Pawelczyk, S. Neel, and H. Lakkaraju, "In-context unlearning: Language models as few shot unlearners," *arXiv preprint arXiv:2310.07579*, 2023.

[10] R. Ye, C. Zhang, R. Wang, S. Xu, and Y. Zhang, "Natural language is all a graph needs," *arXiv preprint arXiv:2308.07134*, 2023.

[11] W. Li, D. Yao, R. Zhao, W. Chen, Z. Xu, C. Gong, Q. Jing, H. Tian and J. Bi*et al.*, "STBench: Assessing the Ability of Large Language Models in Spatio-Temporal Analysis." *arXiv preprint arXiv:2406.19065*, 2024.

[12] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan *et al.*, "Time-llm: Time series forecasting by reprogramming large language models," *arXiv preprint arXiv:2310.01728*, 2023.

[13] X. Xu, K. Ding, C. Chen, and K. Shu, "Metagad: Learning to meta transfer for few-shot graph anomaly detection," *arXiv preprint arXiv:2305.10668*, 2023.

[14] R. Li, X. Wu, X. Wu, and W. Wang, "Few-shot learning for new user recommendation in location-based social networks," in *Proceedings of The Web Conference*, 2020, pp. 2472–2478.

[15] L. Tang and H. Liu, "Relational learning via latent social dimensions," in *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2009, pp. 817–826.

[16] T. Opsahl and P. Panzarasa, "Clustering in weighted networks," *Social networks*, vol. 31, no. 2, pp. 155–163, 2009.

[17] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking graph neural networks for anomaly detection," in *International Conference on Machine Learning*. PMLR, 2022, pp. 21 076–21 089.

[18] Z. Liu, W. Zuo, D. Zhang, and X. Feng, "Rgse: Robust graph structure embedding for anomalous link detection," *IEEE Transactions on Big Data*, 2023.

[19] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2014, pp. 701–710.

[20] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," *arXiv preprint arXiv:2002.07962*, 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on October 20,2026 at 13:28:14 UTC from IEEE Xplore. Restrictions apply.